

1. Configuración y Exploración de Datos (EDA)

Comencé estableciendo un entorno reproducible. Fijé una semilla aleatoria (RANDOM_SEED = 26) en las librerías numpy, random y torch porque en el entrenamiento de redes neuronales es vital asegurar que los resultados sean deterministas y comparables entre diferentes iteraciones.

Al explorar el dataset (1569 registros), me enfoqué en la distribución de las variables de interacción.

- **Decisión:** Grafiqué las variables numéricas (Visits, Likes, Dislikes, etc.) utilizando una escala logarítmica.
- **Por qué:** Este tipo de métricas en plataformas sociales suelen seguir una distribución de cola larga, donde una inmensa mayoría de POIs tiene muy pocas interacciones y una minoría concentra la mayor parte. La escala logarítmica me permitió visualizar estas asimetrías de forma clara.

2. Ingeniería de Características y Definición del Objetivo (Target)

Para transformar este problema en una tarea de clasificación binaria, necesitaba definir qué constituía un "Alto Engagement".

- **La Fórmula:** Diseñé un engagement_score combinando las métricas disponibles: En un principio empezó como un $(\text{Likes} - \text{Dislikes}) + (2 * \text{Bookmarks}) + (\text{Visits})$, pero claro es mucho mas común que haya visitas por lo que decidí penalizarlo bajando al logaritmo multiplicado por 5 quedando así: $\text{Engagement} = (\text{Likes} - \text{Dislikes}) + (2 * \text{Bookmarks}) + (5 * \ln(1 + \text{Visits}))$ Luego. me di cuenta de que las visitas estaban truncadas a 10000 por lo que no tenían casi varianza tenían una std de 5 por lo que decidí simplemente quitarlas de la formula, quedando: $\text{Engagement} = (\text{Likes} - \text{Dislikes}) + (2 * \text{Bookmarks})$.
Por qué: Quería penalizar el rechazo (Dislikes), darle el doble de peso a la retención (Bookmarks demuestra una intención fuerte del usuario).
- **Binarización:** Utilicé la mediana de este score para separar los datos en Alto (1) y Bajo (0). Esto garantizó un problema perfectamente balanceado (50/50), evitando el sesgo hacia una clase mayoritaria, aunque en algún momento también estuve tentado de cambiarlo hacia la derecha para que el engagement tuviese que ser realmente alto para considerarlo bueno.
- **Preprocesamiento Categórico:** Agrupé las categorías con menos de 10 apariciones bajo la etiqueta "Otras". *Por qué:* Los modelos de Machine Learning tienden a sobre ajustarse al ver categorías extremadamente raras, considerándose "ruido". Agruparlas mejora la generalización.
- **Preprocesamiento Visual:** Transformé las imágenes a tamaño de 224x224 y además hice que puedan rotar cambiar el brillo y el contraste en train etcétera para que en cada epoch puedan encontrarse la misma imagen pero modificada, de esta manera no memoriza, tanto y generaliza mejor.

Aprendizaje Crítico sobre el Data Leakage:

Durante este proceso, identifiqué dos errores conceptuales severos de fuga de datos que inflaban el rendimiento inicial a un 97% irreal:

1. **Target Leakage:** El modelo recibía como entrada variables (Likes, Dislikes, Bookmarks, Visits) que matemáticamente componían la variable a predecir. Modifiqué las features numéricas para incluir únicamente aquellas disponibles antes de que el POI reciba visitas (ubicación, número de tags, categoría y nivel).

3. Arquitectura del Modelo: Enfoque Híbrido Multimodal

A. Rama Visual (Extracción de Imágenes)

- **Arquitectura:** ResNet18 preentrenada en ImageNet.
- **Por qué:** Opté por ResNet18 en lugar de versiones más profundas (como ResNet50) para prevenir el sobreajuste (overfitting), dado que mi dataset es relativamente pequeño (~1500 imágenes).
- **Ajuste:** Modifiqué la última capa densa para que proyectara la imagen en un vector de 128 dimensiones en lugar de las 1000 clases originales.

B. Rama Tabular (Metadatos)

- **Arquitectura:** Red neuronal densa (Fully Connected) de dos capas (64 a 32 dimensiones) con activaciones ReLU.
- **Por qué:** Necesitaba proyectar las características numéricas y categóricas para que pudieran relacionarse con las imágenes.
- **Regularización:** Introduce una capa Dropout($p=0.3$). *Por qué:* Para obligar a la red a no depender de ninguna variable tabular en específico y evitar memorizar el conjunto de entrenamiento.

C. Capa de Fusión

- **Arquitectura:** Concatenación del vector visual (128) y el tabular (32), resultando en 160 dimensiones, pasando por una capa oculta de 64 neuronas antes de la salida binaria.
- **Por qué:** Esta fusión permite al modelo aprender correlaciones no lineales complejas (ej. "los POIs de esta categoría rinden mejor cuando la imagen tiene ciertas características de iluminación").

4. Estrategia de Entrenamiento y Resultados Reales

- **Configuración:** Utilicé el optimizador Adam con un learning rate de 0.001, aunque también probe con diferentes valores y CrossEntropyLoss, la combinación estándar y más estable para clasificación en Deep Learning.
- **Control del Entrenamiento:** Implementé un guardado del mejor modelo evaluando la pérdida en el conjunto de validación (best_val_loss). Por qué: Esto es una forma de Early Stopping empírico; garantiza que me quede con los pesos del modelo en su punto de máxima generalización, independientemente de si en épocas posteriores empieza a sobreajustarse.
- **Resultados Finales:** Una vez eliminado el Data Leakage, el modelo estabilizó su precisión en un 83% de Accuracy sobre el set de Test, un resultado sólido, realista y verdaderamente predictivo, lo cual valida que la arquitectura multimodal está logrando abstraer patrones reales entre las imágenes de los lugares y sus metadatos intrínsecos.

5. Futuras mejoras

Como futuras mejoras se me ocurre implementar un embedding para tier y categories así como una posible mejora para poder añadir el campo de short_description basándonos en el texto que nos proporciona.

6.Bibliografía

Para realizar la práctica utilice tanto el contenido dado en clase, como tomé ayuda de Gemini para ayudarme con algunos detalles del código.